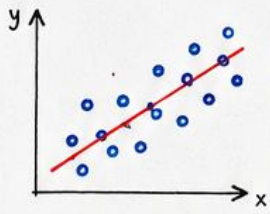
12 ML ALGORITHMS CHEAT SHEET

+ TOP INTERVIEW QUESTIONS

LEARN • PRACTICE • CRACK

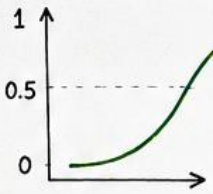
1. Linear Regression



$\hat{y} = wx + b$

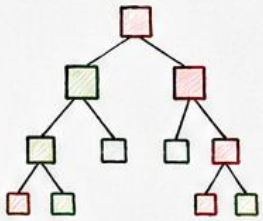
Best for continuous prediction

2. Logistic Regression



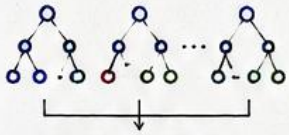
Best for binary classification

3. Decision Tree



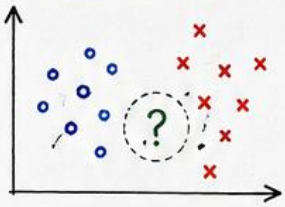
Split data based on feature conditions

4. Random Forest



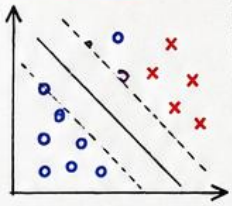
Ensemble of trees reduces overfitting

5. KNN



Classify by majority of nearest neighbors

6. SVM



Finds best hyperplane with maximum margin

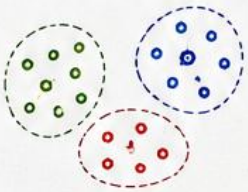
7. Naive Bayes

$$P(C|X) = \frac{P(X|C)\,P(C)}{P(X)}$$

Fast & effective for text classification

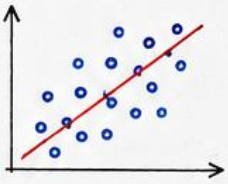


8. K-Means



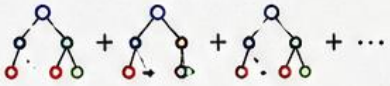
Groups data into K clusters

9. PCA



Reduces dimensions while keeping maximum variance

10. Gradient Boosting / XGBoost

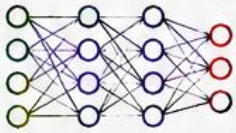


Builds models sequentially, each fixes previous errors

Key Concepts

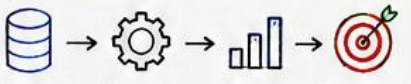
- ☑ Training vs Testing
- ☑ Overfitting vs Underfitting
- ☑ Bias-Variance Tradeoff
- ☑ Feature Scaling
- ☑ Cross-Validation
- ☑ More Data + Good Features = Better Model

11. Neural Networks / Deep Learning



Learns complex patterns using multiple layers

Data Flow



Model Evaluation

- ☑ Accuracy
- ☑ Precision
- ☑ Recall
- ☑ F1-Score
- ☑ ROC-AUC



Loss Functions



Top ML Interview Questions (Categories)

Basics
- What is Machine Learning?
- Types of ML?
- Supervised vs Unsupervised?
- Bias vs Variance?
- Overfitting vs Underfitting?
- Train/Test split?
- Feature Engineering?

Algorithms
- When to use Linear Regression?
- Logistic Regression vs Linear Regression?
- SVM kernel trick?
- KNN: How does it work?
- K-Means limitations?
- Naive Bayes assumptions?

Evaluation
- Precision vs Recall?
- F1 Score?
- ROC AUC?
- Confusion Matrix?
- Cross-Validation?
- Handling Imbalanced Data?

Deep Learning
- ANN vs CNN vs RNN?
- Vanishing/Exploding Gradients?
- Activation functions?
- Backpropagation?
- Transfer Learning?

Real World
- How to handle missing values?
- Outliers detection?
- Feature scaling: why important?
- How to deploy a model?

> The best model is not about algorithms, it's about data, problem & good thinking.

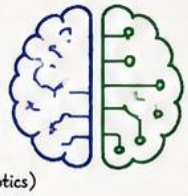
1. ML Basics + Linear Regression

ML Basics

Machine Learning (ML) is a subset of AI that enables systems to learn from data and improve performance without being explicitly programmed.

Types of Machine Learning

- **Supervised Learning** : Learn from labeled data (e.g. Classification, Regression)
- **Unsupervised Learning** : Find patterns in unlabeled data (e.g. Clustering)
- **Reinforcement Learning** : Learn by interacting with environment (e.g. Games, Robotics)



Linear Regression

Linear Regression is a supervised machine learning algorithm used to predict a continuous (numeric) output variable based on one or more input features.

Assumptions of Linear Regression

- ✓ Linearity : Relationship between X and Y is linear
- ✓ Independence : Errors are independent
- ✓ Homoscedasticity : Constant variance of errors
- ✓ Normality : Errors are normally distributed
- ✓ No Multicollinearity : Independent features

Key Equation / Formula :

$$\hat{y} = \beta_0 + \beta_1 x$$

$\hat{y}$ = Predicted value (output)
x = Input feature
β_0 = Intercept (bias)
β_1 = Slope (coefficient)

For multiple features :

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

When to Use :

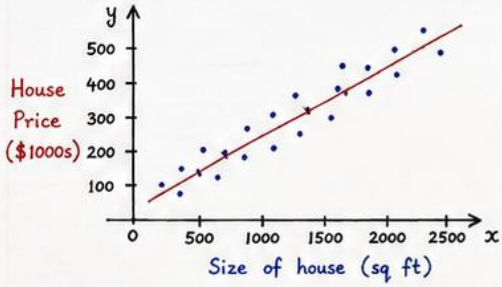
Use Linear Regression when you want to predict a continuous value and the relationship between input feature(s) and output is approximately linear.

How it Works (Architecture) :

Input Features (x) → Linear Model Applies Equation $\hat{y} = \beta_0 + \beta_1 x$ → Prediction ($\hat{y}$) → Model learns best β_0, β_1 to minimize error

The model learns the best values of β_0 and β_1 by minimizing the error between predicted and actual values.

Example / Chart :



Here,
x = Size of house (sq ft)
y = Price of house ($1000s)

The red line is the best fit line predicted by Linear Regression.

Evaluation Metrics :



- ☆ MAE (Mean Absolute Error)
- ☆ MSE (Mean Squared Error)
- ☆ RMSE (Root Mean Squared Error)
- ☆ R^2 Score (Coefficient of Determination)

Pros & Cons :



- ✓ Simple and easy to understand
- ✓ Works well with linear data
- ✓ Fast training and prediction
- ✓ Interpretable results



- ✗ Assumes linear relationship
- ✗ Sensitive to outliers
- ✗ Not good for complex relationships
- ✗ Requires feature scaling in some cases

Real-World Examples :

- Predicting house prices
- Sales forecasting
- Salary prediction
- Temperature prediction





Linear Regression is the foundation of many ML algorithms.
Master it well – it's simple, powerful and widely used! ☺

2. Logistic Regression



What is it?

Logistic Regression is a supervised machine learning algorithm used for binary classification problems.

It predicts the probability that a given input belongs to a particular class (0 or 1).

When to use?

- ☑ When the target variable is binary (0 or 1)
- ☑ When you need probabilities as output
- ☑ When the relationship between features and target is (approximately) linear
- ☑ When the data is linearly separable

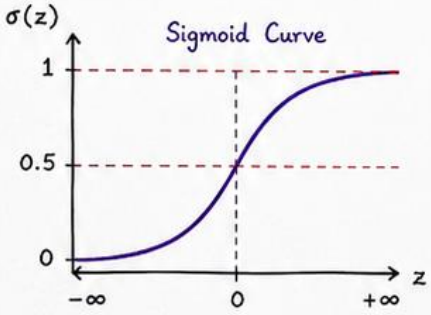


Sigmoid (Logistic) Function

Logistic Regression uses the Sigmoid function to map any real-valued number into a value between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where, $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$



As $z \to -\infty$, $\sigma(z) \to 0$

As $z \to +\infty$, $\sigma(z) \to 1$

At $z = 0$, $\sigma(z) = 0.5$

Model / Hypothesis

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n \rightarrow h_\theta(x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$h_\theta(x)$ = Predicted probability that $y = 1$ given x

If $h_\theta(x) \geq 0.5$ → Predict Class 1

If $h_\theta(x) < 0.5$ → Predict Class 0

Cost Function

Logistic Regression uses Log Loss (Binary Cross-Entropy) to measure the error.

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} [y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))]$$

m = number of training examples

$y^{(i)}$ = actual label (0 or 1)

$h_\theta(x^{(i)})$ = predicted probability

How it Works (Training)

Input Features (X) → Compute Linear Combination (z) → Apply Sigmoid Function σ(z) → Get Probability $h_\theta(x)$ → Compute Cost & Update Parameters

Parameters $(\beta_0, \beta_1, \ldots, \beta_n)$ are learned using Gradient Descent to minimize the cost function.

Example

Predict whether a student will pass (1) or fail (0) based on number of study hours.

Hours (x)	Pass (y)	Predicted $h_\theta(x)$	Prediction
1	0	0.08	Fail (0)
2	0	0.23	Fail (0)
3	0	0.42	Fail (0)
4	1	0.68	Pass (1)
5	1	0.85	Pass (1)
6	1	0.94	Pass (1)

Pros & Cons



- ✓ Simple and easy to implement
- ✓ Outputs probabilities
- ✓ Works well for linearly separable data
- ✓ Efficient and interpretable



- ✗ Not suitable for complex non-linear relationships
- ✗ Sensitive to outliers
- ✗ Assumes linearity between features and log-odds
- ✗ Needs feature scaling for better performance



Key Takeaways

- ☑ Logistic Regression is used for binary classification.
- ☑ It predicts probability using the Sigmoid function.
- ☑ The model is trained by minimizing Log Loss using Gradient Descent.
- ☑ Decision boundary is linear.
- ☑ Threshold (0.5) is used to convert probability to class label.

Real-World Examples



- Email Spam Detection
- Customer Churn Prediction
- Disease Diagnosis (Yes / No)
- Loan Approval Prediction
- Click-Through Rate (CTR) Prediction

3. Decision Trees



What is it?

Decision Tree is a supervised machine learning algorithm used for classification and regression.

It works by splitting the data into subsets based on the most important feature, creating a tree-like structure of decisions.

When to Use?

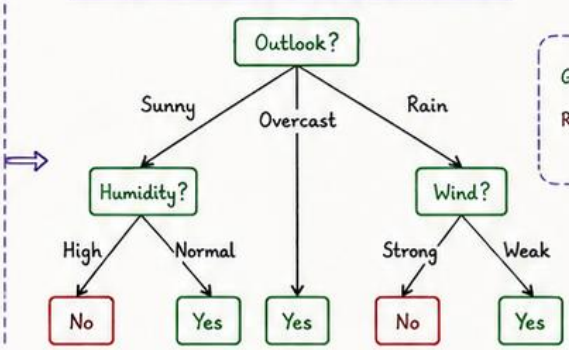
- ☑ When the problem is easy to interpret
- ☑ When you need a model with high interpretability
- ☑ When data has both numerical and categorical features
- ☑ When missing values are present



How it Works?

1. Start from the root node (entire dataset)
2. Find the best feature to split the data
3. Split the data into branches
4. Repeat the process for each subset
5. Stop when nodes are pure or stopping criteria met
6. Leaf nodes make the final prediction

Example Decision Tree (Play Tennis)



Legend

Green box → Decision Node

Red/Green box → Leaf Node (Class)

Example Dataset

Day	Outlook	Temp.	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No

How does it choose the best split?

Decision Trees use metrics like Information Gain (ID3), Gain Ratio (C4.5) or Gini Index (CART) to find the best feature to split.

- **Entropy**

$$H(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

S = Dataset
c = Number of classes
p_i = Proportion of class i

- **Information Gain**

$$IG(S,A) = H(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} H(S_v)$$

A = Feature
v = Value of A
S_v = Subset for v

- **Gini Index**

$$Gini(S) = 1 - \sum_{i=1}^{c} p_i^2$$

Advantages

- ✓ Easy to understand and interpret
- ✓ Works with numerical and categorical data
- ✓ Requires little data preprocessing
- ✓ Can handle non-linear relationships
- ✓ Fast prediction



Disadvantages

- ✗ Prone to overfitting
- ✗ High variance (small changes in data can change the tree)
- ✗ Not good for very complex problems
- ✗ Biased towards features with more levels



Stopping Criteria

- Maximum depth reached
- Minimum samples in a leaf node
- Minimum samples required to split a node
- No improvement in Information Gain / Gini

Pruning

Removing unnecessary branches to reduce overfitting and improve generalization.



Types of Decision Trees

- ID3 → Uses Information Gain
- C4.5 → Uses Gain Ratio
- CART → Uses Gini Index (Classification) and MSE (Regression)

Key Takeaways



- Decision Trees are intuitive and powerful.
- They split data based on the most important features.
- Easy to visualize and explain.
- Prone to overfitting → Use pruning or set max depth.

Real-World Examples



- Loan Approval (Yes / No)
- Customer Churn Prediction
- Medical Diagnosis
- Spam Detection
- Risk Assessment

☆ A good tree is not the biggest tree, but the one that generalizes the best! ☺

4. Random Forest



What is it?

Random Forest is an ensemble learning algorithm that builds multiple Decision Trees during training and outputs the mode (classification) or mean (regression) of the individual trees.

When to use?

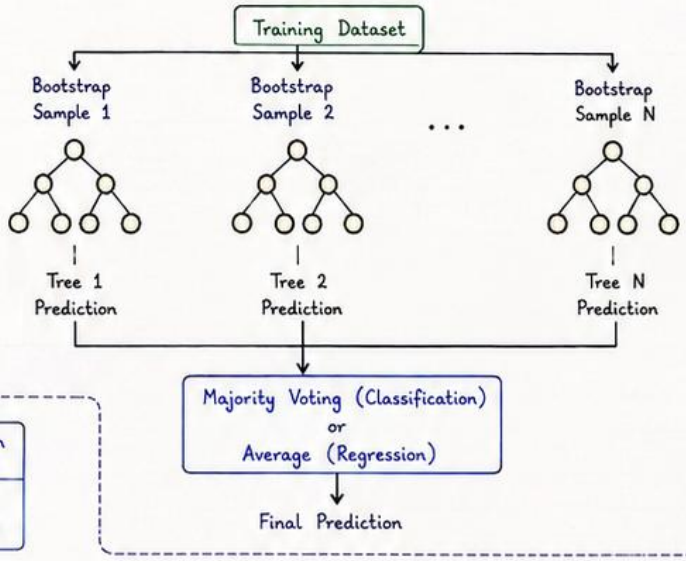
- ☑ When you need high accuracy and robustness
- ☑ When data has non-linear relationships
- ☑ When overfitting is a concern in decision trees
- ☑ When you have large datasets with many features



How it Works?

1. From the original dataset, draw N bootstrap samples (random sampling with replacement).
2. For each sample, grow a Decision Tree.
3. At each split in the tree, consider only a random subset of features.
4. Trees are grown to full depth (no pruning).
5. Aggregate the predictions from all trees.

Random Forest Architecture



Example (Classification)

Tree 1	Tree 2	Tree 3	Tree 4	Tree 5	Final Prediction (Majority Vote)
Class A	Class B	Class A	Class A	Class B	Class A (3 out of 5)

Hyperparameters

- n_estimators : Number of trees in the forest
- max_depth : Maximum depth of each tree
- max_features : Number of features to consider at each split (e.g., sqrt, log2)
- min_samples_split : Minimum samples to split a node
- min_samples_leaf : Minimum samples at a leaf node
- bootstrap : Whether bootstrap samples are used (True / False)

Advantages

- ✓ High accuracy
- ✓ Reduces overfitting (compared to single tree)
- ✓ Handles large datasets well
- ✓ Works with both numerical & categorical data
- ✓ Provides feature importance

Disadvantages

- ✗ Less interpretable than single Decision Tree
- ✗ Requires more computation (time & memory)
- ✗ Not ideal for very high-dimensional sparse data

Out-Of-Bag (OOB) Error

Since each tree is trained on a bootstrap sample, ~1/3 of the data is left out (Out-Of-Bag). These OOB samples can be used to estimate the model error without needing a separate validation set.

How it Reduces Overfitting?

Individual trees may overfit, but averaging many decorrelated trees reduces variance and improves generalization.

Example Dataset

ID	Feature 1	Feature 2	Feature 3	Target
1	5.1	3.5	1.4	0
2	4.9	3.0	1.4	0
3	6.2	3.4	5.4	1
4	5.9	3.0	5.1	1
5	6.3	2.8	5.6	1
...	...			

Random Forest builds many trees and predicts the final class.

Pseudocode

```
1. Input: Training data D, number of trees N
2. For i = 1 to N:
     a. Draw bootstrap sample Dᵢ from D
     b. Grow decision tree Tᵢ on Dᵢ
        - At each split, consider random subset of features
3. For a new instance x:
     - Get prediction from each tree Tᵢ(x)
     - Return majority vote (classification) or mean (regression)
```

Types of Random Forest

- Random Forest Classifier → For classification tasks
- Random Forest Regressor → For regression tasks
- Extra Trees (Extremely Randomized Trees)
  → Uses more randomness, less correlation

More Trees = More Stability Up to a Point!

Real-World Examples

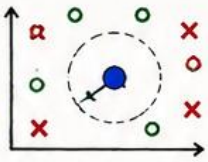


- Disease prediction
- Customer churn prediction
- Fraud detection
- Recommendation systems
- Credit scoring



☆ Random Forest = Many Trees, Smarter Decision! ☺

5. K-Nearest Neighbors (KNN)



What is it?

K-Nearest Neighbors (KNN) is a simple, instance-based learning algorithm used for classification and regression.

It predicts the output based on the majority vote (classification) or average (regression) of the K nearest neighbors in the feature space.

When to Use?

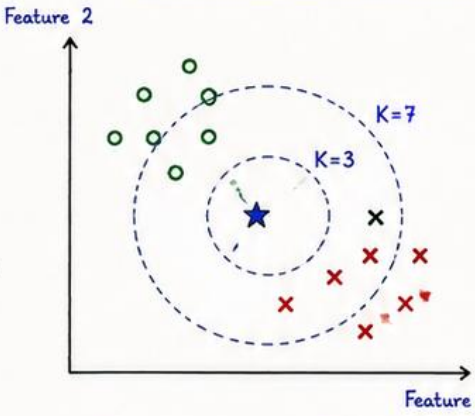
- ☑ When the problem is simple but effective solutions are needed
- ☑ When data is labeled and not too large
- ☑ When the decision boundary is irregular
- ☑ When you have both numerical and categorical data
- ☑ When interpretability is important



How it Works?

1. Choose the value of K.
2. Calculate the distance between the query point and all training points.
3. Select the K nearest neighbors.
4. For classification → Majority vote.
   For regression → Average of outputs.
5. Assign the predicted class/value to the query point.

Example (KNN Classification)



- ○ Class A
- × Class B
- ★ Query Point

For K = 3 → Class A (2 A vs 1 B)

For K = 7 → Class B (4 B vs 3 A)

Distance Metrics

- Euclidean Distance : $d(x,y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$
- Manhattan Distance : $d(x,y) = \sum_{i=1}^{n} |x_i - y_i|$
- Minkowski Distance : $d(x,y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p\right)^{\frac{1}{p}}$

Example Dataset (Classification)

ID	Feature 1	Feature 2	Class
1	2.0	3.0	A
2	3.0	4.0	A
3	5.0	7.0	B
4	6.0	8.0	B
5	7.0	7.0	B

Query Point (2.5, 3.5)

For K = 3

Nearest Neighbors: ID 1 (A), ID 2 (A), ID 3 (B)

Prediction → Class A (Majority Vote)

Hyperparameters

- K : Number of neighbors
- Distance Metric : Euclidean, Manhattan, etc.
- Weights : Uniform or Distance-based
- Algorithm : Auto, Ball Tree, KD Tree, Brute

Advantages

- ✓ Simple to understand and implement
- ✓ No training phase (lazy learner)
- ✓ Works well for non-linear data
- ✓ Effective with small datasets

Disadvantages

- × Slow prediction for large datasets
- × Sensitive to irrelevant features
- × Requires feature scaling
- × Performance depends on value of K

Choosing the Right K

- Small K → Sensitive to noise (overfitting)
- Large K → Smooth decision boundary (underfitting)
- Choose K using cross-validation.

Tip: Use odd K in binary classification to avoid tie votes.



KNN Regression

Instead of majority vote, KNN regression takes the average (or weighted average) of the target values of the K nearest neighbors.

$$\hat{y} = \frac{1}{K} \sum_{i=1}^{K} y_i$$

y_i → target value of the ith nearest neighbor

Types of KNN

- KNN Classifier → For classification problems
- KNN Regressor → For regression problems
- Weighted KNN → Closer neighbors have more influence



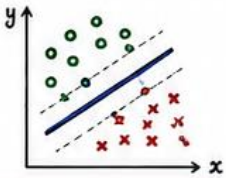
Real-World Examples

- Handwritten digit recognition
- Recommendation systems
- Image classification
- Anomaly detection



☆ KNN is simple but powerful when used with the right data and parameters! ☺

6. Support Vector Machines (SVM)



What is it?

Support Vector Machines (SVM) is a supervised machine learning algorithm used for classification and regression.

SVM finds the optimal hyperplane that best separates the classes by maximizing the margin between them.

When to Use?

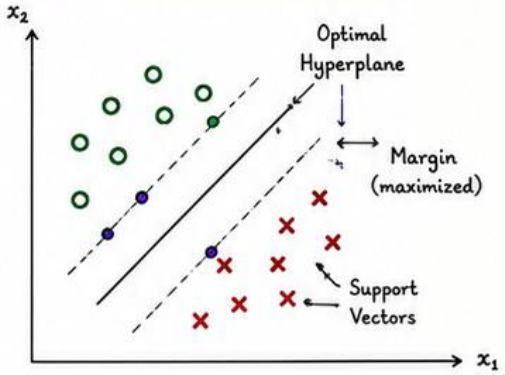
- ☑ When the data is labeled and you need high accuracy
- ☑ When the number of features is high
- ☑ When the classes are clearly separable (or can be made separable using a kernel)
- ☑ When you want a robust model with good generalization



How it Works?

1. Map the input data into a high-dimensional space (if needed) using a kernel function.
2. Find the optimal hyperplane that separates the classes with the maximum margin.
3. The data points closest to the hyperplane are called Support Vectors.
4. New data points are classified based on which side of the hyperplane they fall.
5. For regression, SVM finds a function with maximum margin of tolerance (ε-insensitive loss).

SVM Concept (Classification)



Legend
- Class +1
- Class −1
- Support Vectors

$$\text{Margin} = \frac{2}{\|w\|}$$

w = weight vector

SVM Hyperplane

The optimal hyperplane can be represented as:

$$w^T x + b = 0$$

Decision function:

$$f(x) = \begin{cases} +1 & \text{if } w^T x + b \geq 0 \\ -1 & \text{if } w^T x + b < 0 \end{cases}$$

w = weight vector (normal to hyperplane)
b = bias term

Soft Margin (Slack Variables)

Real-world data may not be perfectly separable. SVM uses slack variables (ξ) to allow some misclassifications.

$$\text{Minimize: } \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$$

$$\text{Subject to: } y_i(w^T x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

C = Regularization parameter (controls trade-off between margin size and classification error)

Kernel Trick

SVM uses kernels to solve non-linear problems by mapping data into a higher-dimensional space.

Common Kernels
- Linear: $K(x, z) = x^T z$
- Polynomial: $K(x, z) = (\gamma x^T z + r)^d$
- RBF (Gaussian): $K(x, z) = e^{-\gamma\|x-z\|^2}$
- Sigmoid: $K(x, z) = \tanh(\gamma x^T z + r)$

γ, r, d = kernel parameters

Example (Classification)

ID	Feature 1	Feature 2	Class
1	2.0	1.0	+1
2	2.5	2.0	+1
3	3.0	1.5	+1
4	6.0	5.5	−1
5	6.5	6.0	−1
6	7.0	5.0	−1

→ SVM finds the best separating hyperplane with maximum margin.

Advantages

- ✓ Works well in high-dimensional spaces
- ✓ Effective in cases of clear margin of separation
- ✓ Uses a subset of training points (Support Vectors)
- ✓ Versatile with different kernels
- ✓ Good generalization



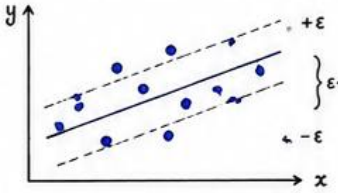
Disadvantages

- × Not suitable for large datasets (slow training)
- × Requires careful tuning of parameters and kernel
- × Performance drops with noisy data and overlapping classes
- × Hard to interpret



SVM Regression (SVR)

SVR tries to fit the best line/curve within a margin of ε.



Only points outside the ε-tube are considered in the loss function.

Real-World Examples

- Image classification
- Handwritten digit recognition
- Text categorization
- Bioinformatics (protein classification)
- Stock market prediction (regression)
- Fault detection



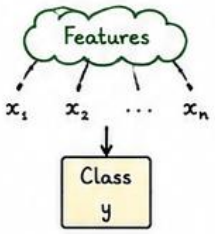


Key Takeaway

SVM aims to find the best boundary that separates the classes with the maximum margin and works great when the margin is clear!



7. Naive Bayes



Assumption: Features are independent given the class.

What is it?

Naive Bayes is a probabilistic machine learning algorithm based on Bayes' Theorem with a "naive" assumption that features are conditionally independent given the class. It is used for classification problems.

When to Use?

- ☑ When data is high-dimensional
- ☑ When features are (approximately) independent
- ☑ When you need a fast and simple baseline model
- ☑ Works well for text data and categorical data



Bayes' Theorem

$$P(C|X) = \frac{P(X|C)\ P(C)}{P(X)}$$

Where,

- C = Class (target)
- X = Feature vector $(x_1, x_2, \ldots, x_n)$
- $P(C|X)$ = Posterior probability (we want this)
- $P(X|C)$ = Likelihood
- $P(C)$ = Prior probability
- $P(X)$ = Evidence (same for all classes)

Naive Assumption

- All features are independent given the class.

$$P(X|C) = \prod_{i=1}^{n} P(x_i|C)$$

Therefore,

$$P(C|X) \propto P(C) \prod_{i=1}^{n} P(x_i|C)$$

Predict class with highest $P(C|X)$

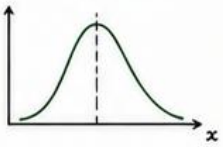
Intuition

We compute the probability of each class given the features and choose the class with highest probability.

Types of Naive Bayes

1) Gaussian NB

- For continuous features assumed to follow normal distribution.



2) Multinomial NB

- For discrete features (e.g., word counts)
- Common in text classification.



3) Bernoulli NB

- For binary features (0 or 1)
- Common in presence/absence data.



Example (Spam Classification)

Email	Contains Money	Contains Win	Class
E1	Yes	No	Spam
E2	No	No	Not Spam
E3	Yes	Yes	Spam
E4	No	Yes	Spam

Predict class of new email:
Contains Money = Yes, Contains Win = No

How it Works?

1. Calculate $P(C)$ for each class.
2. For each feature x_i, calculate $P(x_i|C)$ for each class.
3. Use Naive assumption to get $P(X|C)$.
4. Compute $P(C|X)$ for all classes.
5. Predict the class with highest $P(C|X)$.

Simple Numerical Example

Suppose two classes: Spam (S) and Not Spam (N)

$P(S) = \frac{3}{4}$, $P(N) = \frac{1}{4}$

$P(\text{Money = Yes} \mid S) = \frac{2}{3}$, $P(\text{Win = No} \mid S) = \frac{1}{3}$

$P(\text{Money = Yes} \mid N) = \frac{1}{4}$, $P(\text{Win = No} \mid N) = \frac{3}{4}$

For new email:
Money = Yes, Win = No

$\text{Score}(S) = \frac{3}{4} \times \frac{2}{3} \times \frac{1}{3} = 1/6$

$\text{Score}(N) = \frac{1}{4} \times \frac{1}{4} \times \frac{3}{4} = 3/64$

Since $\frac{1}{6} > \frac{3}{64}$ → Predict Spam

Advantages

- ✓ Simple and easy to implement
- ✓ Fast training and prediction
- ✓ Works well with high-dimensional data
- ✓ Performs well with small datasets
- ✓ Robust to irrelevant features



Disadvantages

- ✗ Naive independence assumption often not true
- ✗ Performance can be poor if features are highly correlated
- ✗ Not suitable for complex relationships



Real-World Examples

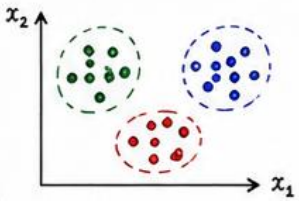
- Email spam filtering
- Sentiment analysis
- Document / text classification
- Medical diagnosis
- Recommendation systems





☆ Naive Bayes is naive in assumption, not in power. ☺

8. K-Means Clustering



What is it?

K-Means is an unsupervised learning algorithm used for clustering.
It groups similar data points into K clusters by minimizing the within-cluster sum of squares (WCSS).

When to Use?

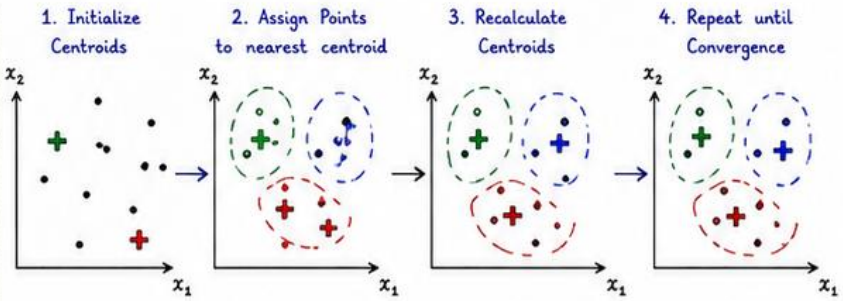
- ☑ When you know the number of clusters (K)
- ☑ When clusters are roughly spherical and similar in size
- ☑ When you have numerical data
- ☑ When speed and scalability are important



How it Works?

1. Choose the number of clusters K.
2. Randomly initialize K centroids.
3. Assign each data point to the nearest centroid (based on distance).
4. Recalculate centroids as the mean of all points in each cluster.
5. Repeat steps 3 and 4 until centroids no longer change (convergence) or max iterations reached.

Working (Example with K = 3)



Objective: Minimize Within-Cluster Sum of Squares (WCSS)

Objective Function (WCSS)

$$WCSS = \sum_{i=1}^{K} \sum_{x \in C_i} ||x - \mu_i||^2$$

Where,

- K = number of clusters
- C_i = set of points in cluster i
- μ_i = centroid (mean) of cluster i
- $||x - \mu_i||^2$ = squared Euclidean distance

Distance Metric

Usually Euclidean Distance:

$$d(x, y) = \sqrt{\sum_{j=1}^{n} (x_j - y_j)^2}$$

- x, y = data points
- n = number of features

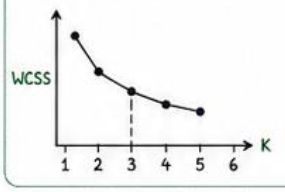
Example Dataset

ID	Feature 1 (x)	Feature 2 (y)
1	2.0	2.0
2	2.5	1.8
3	8.0	8.0
4	8.5	7.5
5	1.0	8.5
6	1.5	7.0

For K = 3, K-Means will group them into 3 clusters (as shown in the example above).

How to Choose K?

- Elbow Method
  Plot WCSS vs K and choose the point where the curve starts to flatten (elbow).



Advantages

- ✓ Simple and easy to implement
- ✓ Works well on large datasets
- ✓ Fast and scalable
- ✓ Often gives good results for compact clusters



Disadvantages

- ✗ Must choose K in advance
- ✗ Sensitive to initial centroids
- ✗ Assumes spherical clusters
- ✗ Sensitive to outliers
- ✗ Doesn't work well with categorical data



Good to Know

- Standardize features before applying K-Means.
- Different initializations can lead to different results.
- Use multiple runs and pick the one with lowest WCSS.



Variants of K-Means

- K-Means++ : Better centroid initialization (chooses distant points)
- Mini-Batch K-Means : Works on small random subsets → faster on big data
- Fuzzy C-Means : Soft clustering (points can belong to multiple clusters)



Real-World Examples



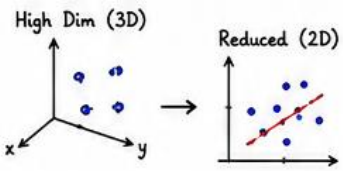
- Customer segmentation
- Image compression (color quantization)
- Market basket analysis
- Document clustering
- Anomaly detection (as a preprocessing step)

Pseudocode

1. Input: Data points X, number of clusters K
2. Initialize K centroids randomly
3. Repeat until convergence:
   a. Assign each point to the nearest centroid
   b. Update each centroid (mean of points in the cluster)
4. Output: Final clusters and centroids

☆ K-Means is simple, fast and powerful for discovering natural groups in data! ☺

9. PCA + Dimensionality Reduction



What is it?

PCA (Principal Component Analysis) is an unsupervised technique used for dimensionality reduction.

It transforms high-dimensional data into a lower-dimensional space while preserving the maximum possible variance (information).

When to use?

- ☑ When data has too many features (high dimensionality)
- ☑ When features are correlated / redundant
- ☑ When you want to visualize high-dimensional data (2D/3D)
- ☑ When you want to reduce noise and computation
- ☑ As a preprocessing step before ML models

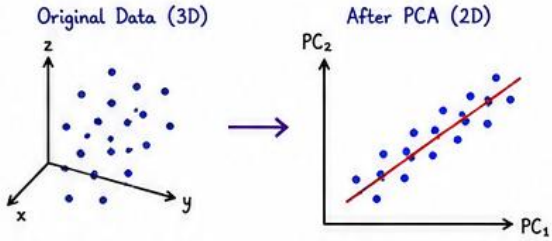


How does PCA Work?

1. Standardize the data (mean = 0, variance = 1)
2. Compute the covariance matrix of the data
3. Compute eigenvalues and eigenvectors
4. Sort eigenvectors by descending eigenvalues
5. Choose top k eigenvectors (principal components)
6. Project the data onto new k-dimensional space

Idea (Intuition)

PCA finds new axes (principal components) that capture the most variance.



We keep the important information (variance) and drop the rest.

Mathematics

Data Matrix: $X \in \mathbb{R}^{n \times d}$ (n samples, d features)

- Covariance Matrix:

$$\Sigma = \frac{1}{n-1} X^T X$$

- Eigen Decomposition:

$$\Sigma v_i = \lambda_i v_i$$

where λ_i = eigenvalue (variance along v_i)
v_i = eigenvector (principal component)

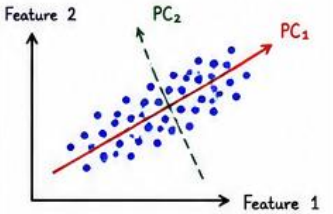
- Projection (to k-dim):

$$Z = X \cdot W_k$$

where $W_k = [v_1, v_2, \ldots, v_k]$ (top k eigenvectors)

Example

Suppose we have 2 features.



PC1 → direction of maximum variance

PC2 → direction of second maximum variance (orthogonal to PC1)

Example Dataset

ID	Feature 1	Feature 2
1	2.5	2.4
2	0.5	0.7
3	2.2	2.9
4	8.0	8.1
5	8.3	7.6
...	...	...

→

After PCA (k = 1)

PC1
-2.0
-3.1
-1.8
3.9
3.8
...

(Data reduced from 2D to 1D)

Advantages

- ✓ Reduces dimensionality
- ✓ Keeps most of the information
- ✓ Removes noise and redundancy
- ✓ Improves visualization
- ✓ Speeds up ML models



Disadvantages

- ✗ PCA is linear (may not capture non-linear patterns)
- ✗ Information loss (when reducing dimensions)
- ✗ Not good if important variance is in lower components



Good to Know

- PCA is unsupervised (uses no labels)
- Components are orthogonal (uncorrelated)
- Explained Variance Ratio tells how much information each component keeps.



Explained Variance Ratio

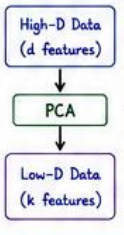
Tells how much variance is captured by each principal component.

$$EVR_i = \frac{\lambda_i}{\sum_{j=1}^{d} \lambda_j}$$

Choose k such that cumulative EVR is high (e.g., > 90%)

PCA Workflow (Summary)

1. Standardize data
2. Compute covariance matrix
3. Compute eigenvalues & eigenvectors
4. Sort by eigenvalues (high to low)
5. Pick top k components
6. Transform data to new subspace



Real-World Examples

- Face recognition (eigenfaces)
- Image compression
- Text data visualization
- Genomics (gene expression data)
- Finance (portfolio risk factors)



Tiny Numerical Example

If eigenvalues = [4.0, 1.0]
Total variance = 5.0

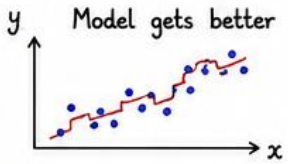
- Using 1 component (PC1):
  $EVR_1 = 4.0 / 5.0 = 0.80$ (80%)
- Using 2 components:
  EVR = 1.00 (100%)

So with 1 component we keep 80% of the information!



☆ PCA helps us see the big picture by focusing on what matters most! ☺

10. Gradient Boosting + XGBoost



What is it?

Gradient Boosting is an ensemble learning technique that builds models sequentially. Each new model tries to correct the errors (residuals) made by the previous models.

XGBoost (eXtreme Gradient Boosting) is an optimized and regularized implementation of Gradient Boosting.

When to Use?

- ☑ When you need high predictive accuracy
- ☑ When you have structured/tabular data
- ☑ When relationships are complex (non-linear)
- ☑ When feature interactions are important
- ☑ When you want a strong baseline model for competitions



How Gradient Boosting Works (Regression)

1. Start with a simple model $F_0(x)$ (e.g., mean of y).
2. Compute residuals (errors).

$$r_1 = y - F_0(x)$$

3. Train a weak learner $h_1(x)$ to predict residuals r_1.
4. Update model: $F_1(x) = F_0(x) + \eta\, h_1(x)$

(η = learning rate)

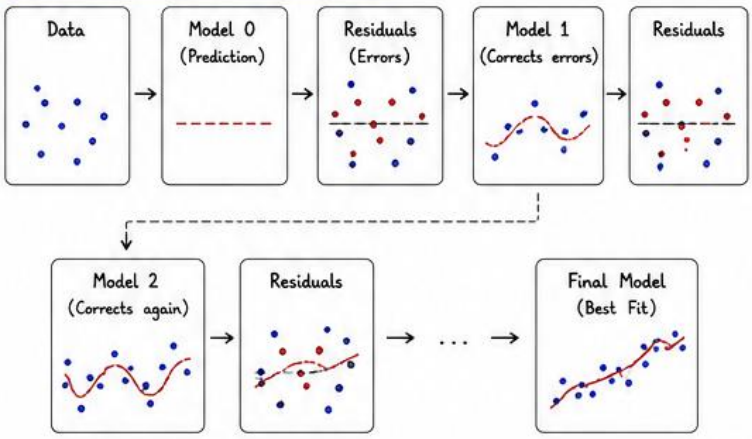
5. Compute new residuals: $r_2 = y - F_1(x)$
6. Train $h_2(x)$ on r_2 and update:

$$F_2(x) = F_1(x) + \eta\, h_2(x)$$

7. Repeat for M iterations.

Final model: $F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} h_m(x)$

Sequential Learning (Visual)



Each model focuses on the mistakes of the previous models.

XGBoost – Key Highlights

- ✓ Regularization (L1 & L2) to prevent overfitting
- ✓ Handles missing values internally
- ✓ Tree pruning using depth & leaf weights
- ✓ Parallel processing (faster training)
- ✓ Built-in cross-validation
- ✓ Works well with small to large datasets



Objective Function (XGBoost)

XGBoost minimizes the regularized objective:

$$Obj = \sum_{i=1}^{n} \ell(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

Where,

ℓ = loss function (e.g., squared error, log loss)

f_k = k-th tree

$\Omega(f_k) = \gamma T + \frac{1}{2}\lambda \|w\|^2$

T = number of leaves, w = leaf weights

γ = complexity penalty, λ = L2 regularization

Example – Regression

x	y (Actual)	Pred $F_0(x)$ (Mean)	Residual (r_1)
1	2.0	4.6	-2.6
2	3.0	4.6	-1.6
3	4.0	4.6	-0.6
4	6.0	4.6	1.4
5	8.0	4.6	3.4

Train $h_1(x)$ on residuals.

Update: $F_1(x) = 4.6 + \eta\, h_1(x)$

Repeat...

Advantages

- ✓ Very high predictive performance
- ✓ Captures complex patterns
- ✓ Robust to overfitting (with regularization)
- ✓ Handles different data types
- ✓ Provides feature importance



Disadvantages

- ✗ More hyperparameters to tune
- ✗ Sensitive to noisy data/outliers
- ✗ Training can be slower (compared to RF)
- ✗ Needs careful tuning



Important Hyperparameters

- n_estimators : Number of trees
- learning_rate (η): Shrinks contribution of each tree
- max_depth : Maximum depth of trees
- subsample : Fraction of data samples
- colsample_bytree : Fraction of features
- gamma : Minimum loss reduction
- lambda : L2 regularization



Good to Know

- Small learning_rate + large n_estimators usually works best.
- Use early stopping to avoid overfitting.
- XGBoost is the go-to algorithm for many ML competitions!



XGBoost vs Others

Model	Speed	Accuracy	Overfitting	Handles Missing	Interpretability
Linear Regression	Fast	Medium	Low	No	High
Random Forest	Medium	High	Medium	Yes	Medium
Gradient Boosting	Slow	Very High	Medium	Yes	Medium
XGBoost	Faster	Very High	Low (with reg.)	Yes	Medium

Real-World Examples

- Kaggle competitions (house prices, fraud, churn)
- Click-through rate (CTR) prediction
- Credit scoring & risk analysis
- Demand forecasting
- Recommendation systems





GRADIENT BOOSTING builds models sequentially to fix errors.
XGBOOST makes it faster, regularized, and production-ready!



11. Neural Networks + Deep Learning Basics



What is it?

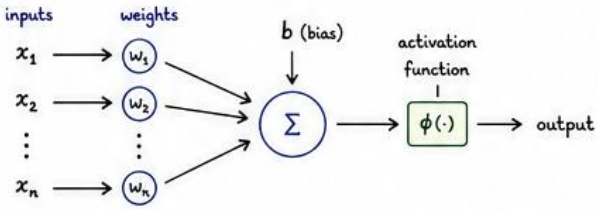
Neural Networks (NNs) are a class of machine learning models inspired by the human brain. They learn patterns from data using layers of interconnected nodes (neurons).
Deep Learning uses NNs with multiple hidden layers (deep networks) to learn complex representations automatically.

When to Use?

- ☑ Large amount of data
- ☑ Complex patterns / high-dimensional data (images, text, audio)
- ☑ When feature engineering is hard
- ☑ When you need state-of-the-art performance
- ☑ When you have computational resources (GPU/TPU)



Neuron (The Basic Unit)



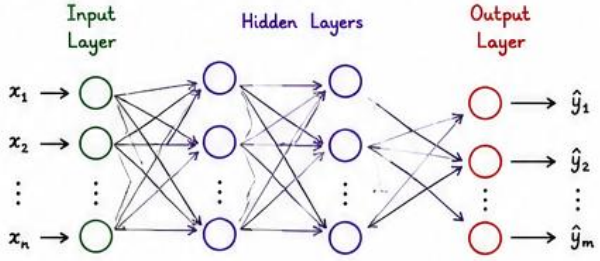
$$z = \sum_{i=1}^{n} w_i x_i + b \quad \text{(weighted sum)}$$

$$a = \phi(z) \quad \text{(activation)}$$

Common activations:

- Sigmoid: $\phi(z) = \frac{1}{1+e^{-z}}$
- Tanh: $\phi(z) = \tanh(z)$
- ReLU: $\phi(z) = \max(0, z)$
- Leaky ReLU: $\phi(z) = \max(\alpha z, z)$

Neural Network Architecture



Input Neuron ○ Hidden Neuron ○ Output Neuron

Information flows forward (left → right).

How does it Learn? (Training)

1. **Forward Pass:** Input goes through the network to get prediction.
2. **Compute Loss:** Measure error (how far prediction is from truth).
3. **Backward Pass:** Compute gradients of loss w.r.t. parameters using Backpropagation.
4. **Update Weights:** Adjust weights to reduce loss (Gradient Descent).
5. Repeat for many epochs until model converges.

Backpropagation in a Nutshell
Gradients flow backward from output to input, telling each weight how much it contributed to the error.
← Error signal flows backward

Loss Functions (Examples)

- Regression: $\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
- Binary Classification (BCE):

$$L = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

- Multi-class Classification (Cross-Entropy):

$$L = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\log(\hat{y}_{ik})$$

(y: true label, $\hat{y}$: predicted probability)

Optimizers

- SGD (Stochastic Gradient Descent)
- Momentum
- RMSprop
- Adam (most widely used)
- AdamW



Adam is a combination of Momentum + RMSprop.

Key Hyperparameters

- Learning Rate (η)
- Batch Size
- Number of Layers
- Number of Neurons
- Activation Functions
- Optimizer
- Dropout Rate
- Regularization (L1/L2)



Common Regularization Techniques

- Dropout: randomly drop neurons during training.
- L2 Regularization (Weight Decay): add $\lambda \|W\|_2^2$ to loss.
- Early Stopping: stop when validation loss stops improving.
- Batch Normalization: stabilize and speed up training.



Types of Neural Networks

- Feedforward Neural Networks (FNN)
- Convolutional Neural Networks (CNN) – for images, computer vision
- Recurrent Neural Networks (RNN) – for sequences, time series, NLP
- Long Short-Term Memory (LSTM) & GRU – advanced RNNs for long-term dependencies
- Transformers – attention based models (state-of-the-art in NLP, etc.)

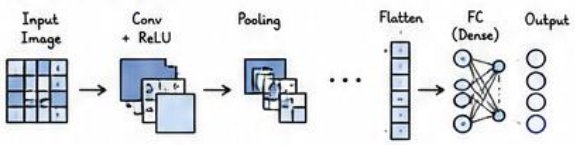


Good to Know

- ☆ More data + compute usually helps.
- ☆ Normalize your data.
- ☆ Start simple, then make it complex.
- ☆ Overfitting is the enemy!

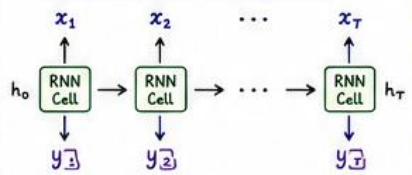


CNN (Quick View)



CNNs learn spatial features using filters.

RNN / LSTM (Quick View)



Good for sequential data (text, speech, time series).

Deep Learning Stack

- Frameworks: TensorFlow, Keras, PyTorch, JAX
- Hardware: GPU / TPU
- Tools: CUDA, cuDNN



☆ Deep Learning = Representation Learning + Big Data + Compute Power ☺
Practice, Experiment, Repeat!

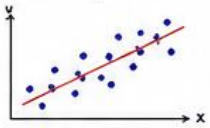
12. Quick Revision Cheat Sheet + Top ML Interview Questions

Keep it Simple.
Understand Clearly.
Crack Confidently. ☺

ML Algorithms – One Page Summary

1. Linear Regression

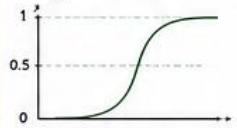
Predicts continuous output using a linear relationship.



Best For: Trend prediction, numeric value prediction

2. Logistic Regression

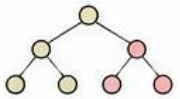
Classifies output into 2 classes using the sigmoid function.



Best For: Binary classification

3. Decision Trees

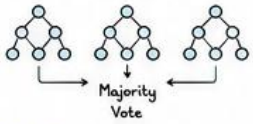
Splits data based on feature conditions in a tree structure.



Best For: Easy to interpret classification & regression

4. Random Forest

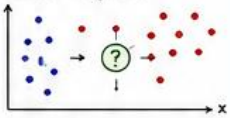
Ensemble of decision trees. Reduces overfitting using bagging.



Best For: High accuracy on tabular data

5. KNN

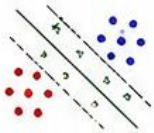
Predicts based on majority vote (classification) or average (regression) of nearest neighbors.



Best For: Simple, non-linear problems

6. SVM

Finds the best hyperplane that maximizes the margin between classes.



Best For: High dimensional data, clear margin

7. Naive Bayes

Uses Bayes' theorem with independence assumption between features.

$$P(C|X) = \frac{P(X|C)\ P(C)}{P(X)}$$

Best For: Text classification, spam detection

8. K-Means Clustering

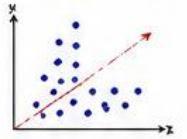
Groups data into K clusters by minimizing within-cluster sum of squares (WCSS).



Best For: Unsupervised clustering

9. PCA

Reduces dimensionality by keeping principal components with maximum variance.



Best For: Visualization, noise reduction

10. Gradient Boosting / XGBoost

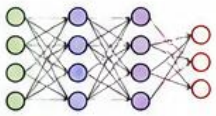
Builds trees sequentially, each new tree corrects previous errors.

Tree 1 → Tree 2 → ··· → Tree n

Best For: High performance on structured data

11. Neural Networks / Deep Learning

Uses multiple layers (neurons) to learn complex patterns automatically.



Best For: Images, NLP, sequences, complex data

Important Formulas

- Mean (μ) = $\frac{1}{n}\sum_{i=1}^{n} x_i$
- Variance (σ^2) = $\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$
- Standardization: $z = \frac{x - \mu}{\sigma}$
- Bayes' Theorem: $P(C|X) = \frac{P(X|C)\ P(C)}{P(X)}$
- Euclidean Distance:

$$d(x, y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

- MSE = $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

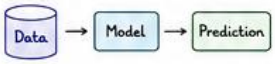
Evaluation Metrics

- Regression:
  - MAE = $\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$
  - MSE = $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
  - $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$
- Classification:
  - Accuracy = $\frac{TP + TN}{TP + TN + FP + FN}$
  - Precision = $\frac{TP}{TP + FP}$
  - Recall = $\frac{TP}{TP + FN}$
  - F1 Score = $2 \times \frac{(Precision \times Recall)}{(Precision + Recall)}$

Actual \ Pred	Positive	Negative
Positive	TP	FN
Negative	FP	TN

ML Workflow (In Short)

1. Problem Definition
2. Data Collection
3. Data Cleaning
4. Exploratory Data Analysis (EDA)
5. Feature Engineering
6. Train/Validation/Test Split
7. Model Selection
8. Training
9. Evaluation
10. Hyperparameter Tuning
11. Prediction
12. Deployment & Monitoring



Key Concepts to Remember

- ✓ Overfitting: Model learns noise
- ✓ Underfitting: Model too simple
- ✓ Bias: Error due to wrong assumptions
- ✓ Variance: Error due to sensitivity to data
- ✓ Train ≠ Test performance
- ✓ Feature Scaling helps many algorithms (SVM, KNN, NN, etc.)
- ✓ Cross-Validation gives robust performance estimate
- ✓ More data + good features + right model = Better performance

Top ML Interview Questions

Basics

1. What is Machine Learning?
2. What is the difference between ML, AI and Deep Learning?
3. What are the types of Machine Learning?
4. What is overfitting and how to handle it?
5. What is underfitting?
6. Bias vs Variance – explain with example.
7. What is the curse of dimensionality?
8. What is the difference between supervised and unsupervised learning?
9. What is feature engineering?
10. What is the train/validation/test split and why?

Regression

11. What is Linear Regression? Assumptions?
12. What is cost function in Linear Regression?
13. What is R^2 score? How is it interpreted?
14. What is the difference between MAE, MSE, RMSE?
15. When will you use Ridge and Lasso Regression?

Classification

16. What is Logistic Regression? How does it work?
17. What is the difference between Precision and Recall?
18. What is the F1 Score and when to use it?
19. What is ROC-AUC?
20. What is the difference between Type I and Type II errors?
21. How does Naive Bayes work? When to use it?
22. What is the difference between KNN and K-Means?
23. What is SVM? How does it work?
24. What is the kernel trick in SVM?

Trees & Ensemble

25. How does a Decision Tree work?
26. What is the difference between Gini and Entropy?
27. What is Random Forest? Why is it better?
28. What is the difference between Bagging and Boosting?
29. How does Gradient Boosting work?
30. What is XGBoost? Why is it popular?

Unsupervised & Dimension Reduction

31. What is K-Means? How does it work?
32. How do you choose the value of K in K-Means?
33. What is PCA? How does it work?
34. What is the difference between PCA and LDA?
35. When will you use PCA?

Deep Learning

36. What is a Neural Network?
37. What is the difference between ANN, CNN and RNN?
38. What is the vanishing gradient problem?
39. What is dropout and why is it used?
40. What are activation functions? Which ones have you used?
41. What is backpropagation?
42. What are optimizers? Which one do you prefer?

Situational / Real-World

43. How will you handle missing values?
44. How will you handle imbalanced data?
45. How will you select important features?
46. How will you tune hyperparameters?
47. How will you deploy an ML model in production?
48. How will you monitor a model after deployment?
49. How will you improve a model that is performing poorly?

Data & Practical

50. How do you handle outliers?
51. What is normalization vs standardization?
52. How do you detect multicollinearity?
53. What is a confusion matrix?
54. How do you handle categorical variables?
55. What is cross-validation? Why is it used?
56. How do you deal with large datasets?

Bonus (Advanced)

57. Explain the bias-variance tradeoff in detail.
58. What is ensembling? Give real examples.
59. How does XGBoost handle overfitting?
60. What is the difference between correlation and causation?

☆ Understand Concepts. Practice Problems. Build Projects. Stay Consistent. You've Got This! ☺